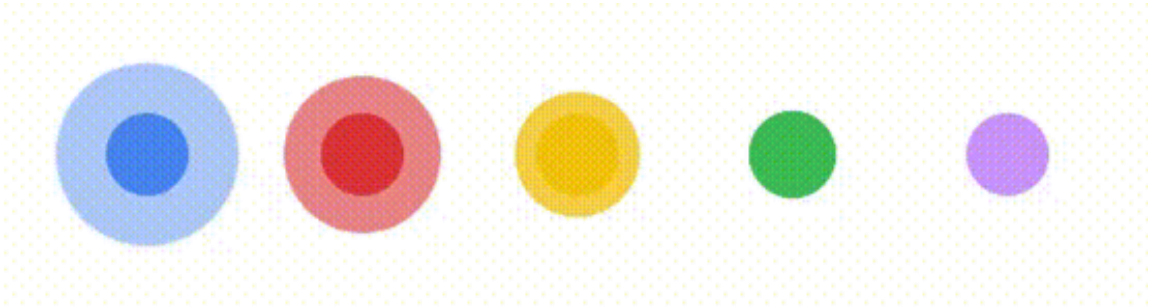
Web Programming Lab

DOM, Event and Form

Week 4

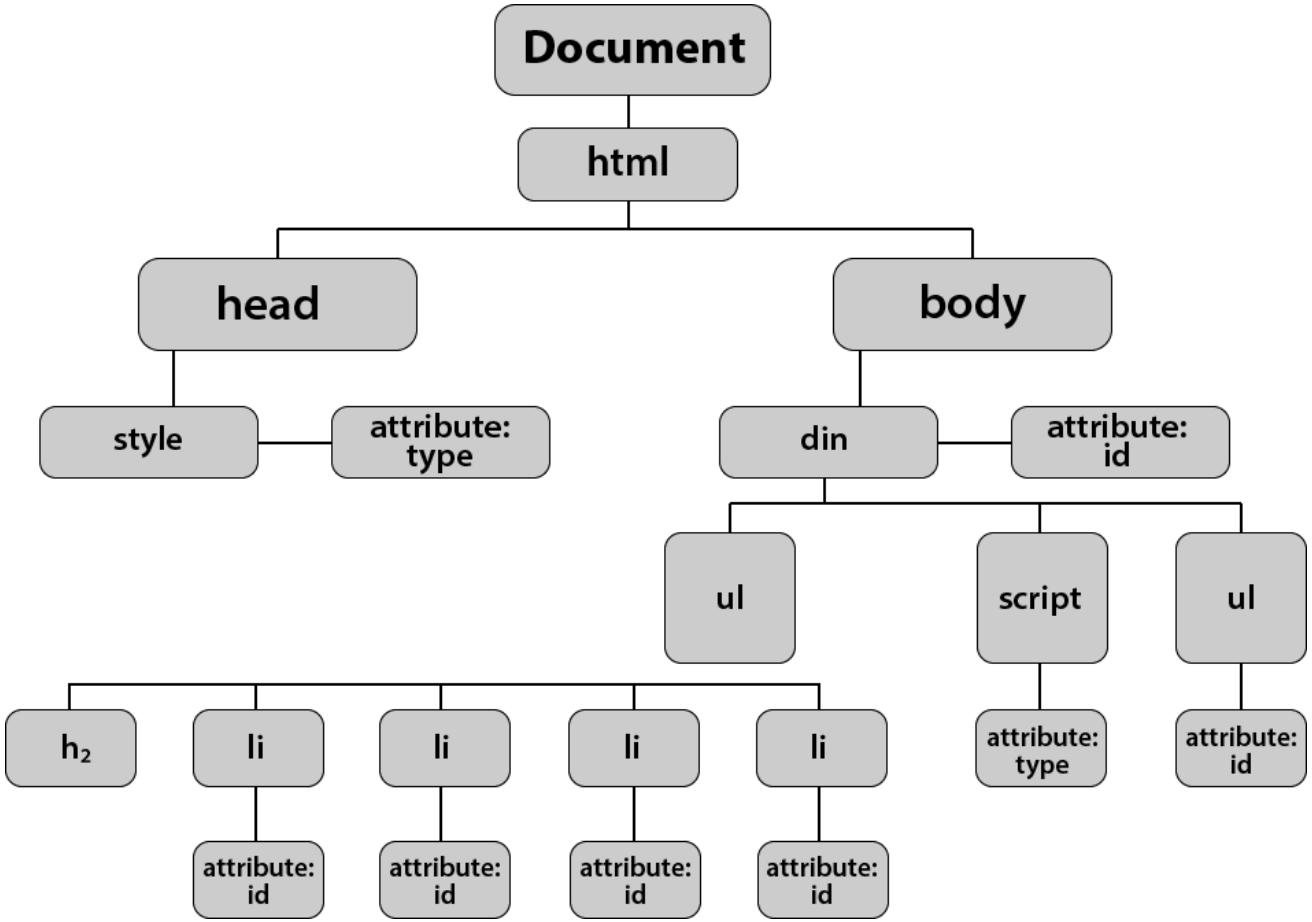


DOM, Event and Form

Document Object Model (DOM)

- JS can access the web page using the DOM
  - Each element is an Element (which is also a Node)
  - Can walk the tree and add/change/remove elements
- Builtin variables
  - window: info/control the browser window
  - document: access the DOM
    - document.head, document.body

Graphic Representation of DOM



Accessing Elements in DOM

Gets	Selector Syntax	Method
ID	#demo	getElementById()
Class	.demo	getElementsByClassName()
Tag	demo	getElementsByTagName()
Selector (single)		querySelector()
Selector (all)		querySelectorAll()

Accessing Elements in DOM

<index.html>

```
<!DOCTYPE html>
<body>
    <button id="clickme">clickme</button>
</body>
<script src="./script.js"></script>
```

<script.js>

```
const btn =
document.getElementById("clickme");
console.log(btn);
```

Node Relationship

- What's the parentNode of ?

```
<body>
    <h1>Heading 1</h1>
    <div>
        <p>Paragraph</p>
        <p>Another Paragraph</p>
    </div>
</body>
```

Node Relationship

- What's the parentNode of ?
- Answer : <body>

```
<body>
    <h1>Heading 1</h1>
    <div>
        <p>Paragraph</p>
        <p>Another Paragraph</p>
    </div>
</body>
```

Node Relationship

- How many childNodes does have?

```
<body>
    <h1>Heading 1</h1>
    <div>
        <p>Paragraph</p>
        <p>Another Paragraph</p>
    </div>
</body>
```

Node Relationship

- How many childNodes does have?
- Answer : 5
  - Why?

```
<body>
    <h1>Heading 1</h1>
    <div>
        <p>Paragraph</p>
        <p>Another Paragraph</p>
    </div>
</body>
```

Node Relationship

- **Whitespace inside elements is considered as text, and text is considered as nodes. Comments are also considered as nodes.**

<index.html>

```
<body>
    <h1>Heading 1</h1>
    <div>
        <p>Paragraph</p>
        <p>Another Paragraph</p>
    </div>
    <button onclick="myFunction()">Check childNodes</button>
    <p id="demo"></p>
    <script src="script.js"></script>
</body>
```

Node Relationship

<script.js>

```
function myFunction() {
    let c = document.getElementsByTagName(“div")[0].childNodes;
    let txt = "";
    let i;
    for (i = 0; i < c.length; i++) {
        txt = txt + c[i].nodeName + "<br>";
    }
    document.getElementById("demo").innerHTML = txt;
}
```

Result


127.0.0.1:5500/week4.html
127.0.0.1:5500/week4.html
Heading 1
Paragraph
Another Paragraph
Check childNodes
#text
P
#text
P
#text

Events

- Javascript can be executed when an event occurs
- To execute code when a user clicks on an element, add JavaScript code to an HTML event attribute

<index.html>

```
<!DOCTYPE html>
<html>
    <body>
        <h1 onclick="this.innerHTML='Got you!'">Click on this text!</h1>
    </body>
</html>
```

HTML Interactors

- <button>: a button
  - Best practice: don't use <input type="button">
  - Children can be anything (text, images)
- <input>: get user input
  - Leaf element (no closing tag)
  - type determines input type (default to text)
  - text, checkbox, radio
- <label>: label an input
  - Wrap the <input> or use for attribute with an id

Handling Events

- elem.addEventListener(type, fn)
  - type is the event to handle (e.g. click)
  - fn is a function to handle the event
  - Note: functions can be passed as values!
- Event types
  - Mouse: click, mouseenter, mouseleave
  - Keyboard: keydown, keyup, keypress
  - Interaction: change, input, focus, blur

Handling Events & Adding Element

<index.html>

```
<!DOCTYPE html>
<head>
</head>
<body>
    <button id="clickme">clickme</button>
    <div id="content-container">
    </div>
</body>
<script src="./script.js"></script>
```

<script.js>

```
const handleClick = (event) => {
    alert("Button was clicked!");
    const element = document.getElementById("content-container");
    console.log(element);
    element.innerHTML="Button was clicked!";
};

const btn = document.getElementById("clickme");
btn.addEventListener("click", handleClick);
```

Handling Events & Adding Element

<index.html>

```
<!DOCTYPE html>
<head>
</head>
<body>
    <button id="clickme">clickme</button>
    <div id="content-container">
    </div>
</body>
<script src="./script.js"></script>
```

<script.js>

```
const handleClick = (event) => {
    alert("Button was clicked!");
    const para = document.createElement("p");
    const node = document.createTextNode("Button was clicked!");
    para.appendChild(node);
    const element = document.getElementById("content-container");
    console.log(element);
    element.appendChild(para);
}

const btn = document.getElementById("clickme");
btn.addEventListener("click", handleClick);
```

HTML Forms

- <form>: wrap inputs/buttons
  - Historically used to send data to server
- Inside a form
  - <button> defaults to submit (use type="button" to override)
  - <input name="foo">
    - Can use instead of id (unique within form but not whole page)
  - <button type="reset">
  - formElem.reset()
    - Clear all inputs in the form
- We will deal with "Form" when we can implement backend server

Weekly Assignment

- Implement login function using JS
  - But we don't have account database..
- Just pop up alert message if id == 'admin' and password == '1234'
- Compress your folder and submit it